CLASS ASSINGMENT 8
DATE : 15-10-25
ID : 12341550

Q1

part1:

code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem;

void* threadA(void* arg)
{
  printf("Hello from A\n");
  sem_post(&sem);
  return NULL;
}
void* threadB(void* arg)
{
  sem_wait(&sem);
  printf("Hello from B\n");
  return NULL;
}
int main()
{
  pthread_t tA, tB;

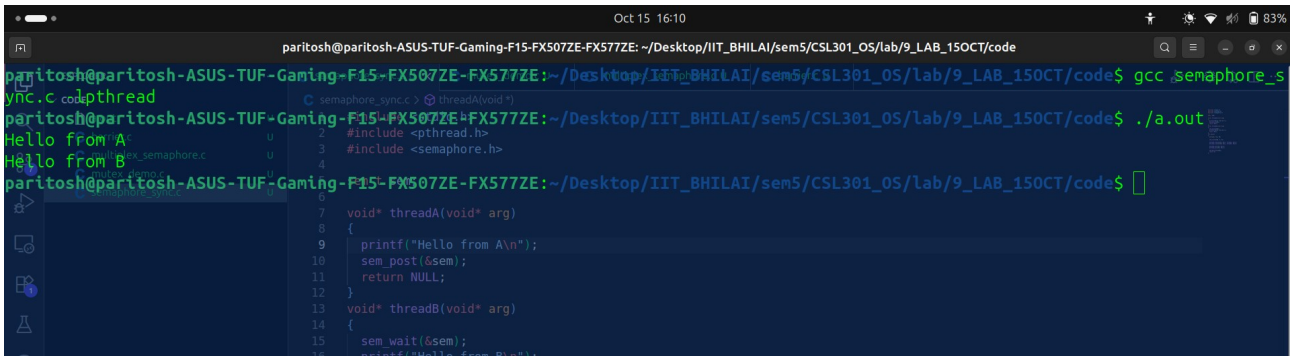
  sem_init(&sem, 0, 0);
```

```
    pthread_create(&tA, NULL, threadA, NULL);
    pthread_create(&tB, NULL, threadB, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    sem_destroy(&sem);
    return 0;
  }
```

output:



observation:
Here order is maintain first A then B is printed due to semaphores...without A , B can not be printed.

part2:

code:

```
  #include <stdio.h>
  #include <pthread.h>
  #include <semaphore.h>
```

```
void* threadA(void* arg)
{
  printf("Hello from A\n");

  return NULL;
}
void* threadB(void* arg)
{

  printf("Hello from B\n");
  return NULL;
}
int main()
{
  pthread_t tA, tB;

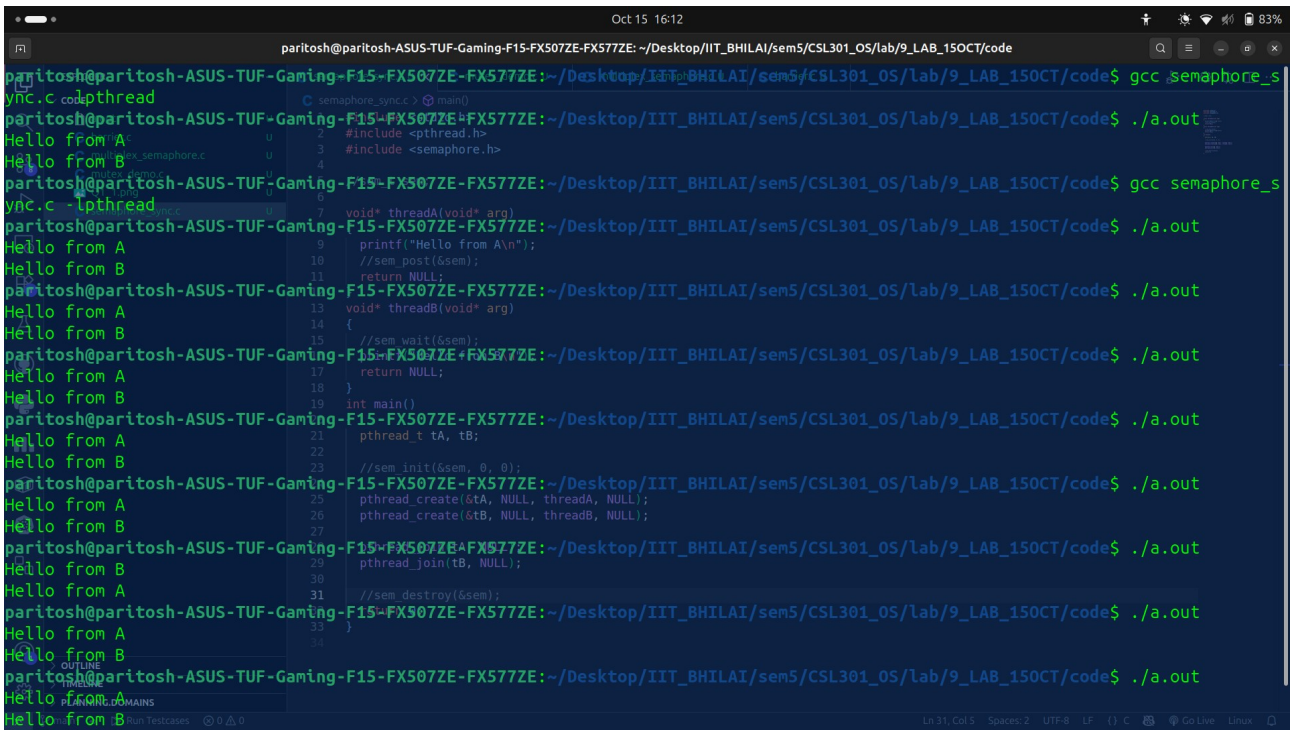


  pthread_create(&tA, NULL, threadA, NULL);
  pthread_create(&tB, NULL, threadB, NULL);

  pthread_join(tA, NULL);
  pthread_join(tB, NULL);


  return 0;
}
```

ouput:



observation:
here order is not maintained , A or B any can print first.

part3:

code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem1,sem2;

void* threadA(void* arg)
{
  printf("Hello from A\n");
  sem_post(&sem1);
  return NULL;
}
void* threadB(void* arg)
```

```
{
  sem_wait(&sem1);
  printf("Hello from B\n");
  sem_post(&sem2);
  return NULL;
}
void* threadC(void* arg)
{
  sem_wait(&sem2);
  printf("Hello from B\n");
  return NULL;
}
int main()
{
  pthread_t tA, tB;

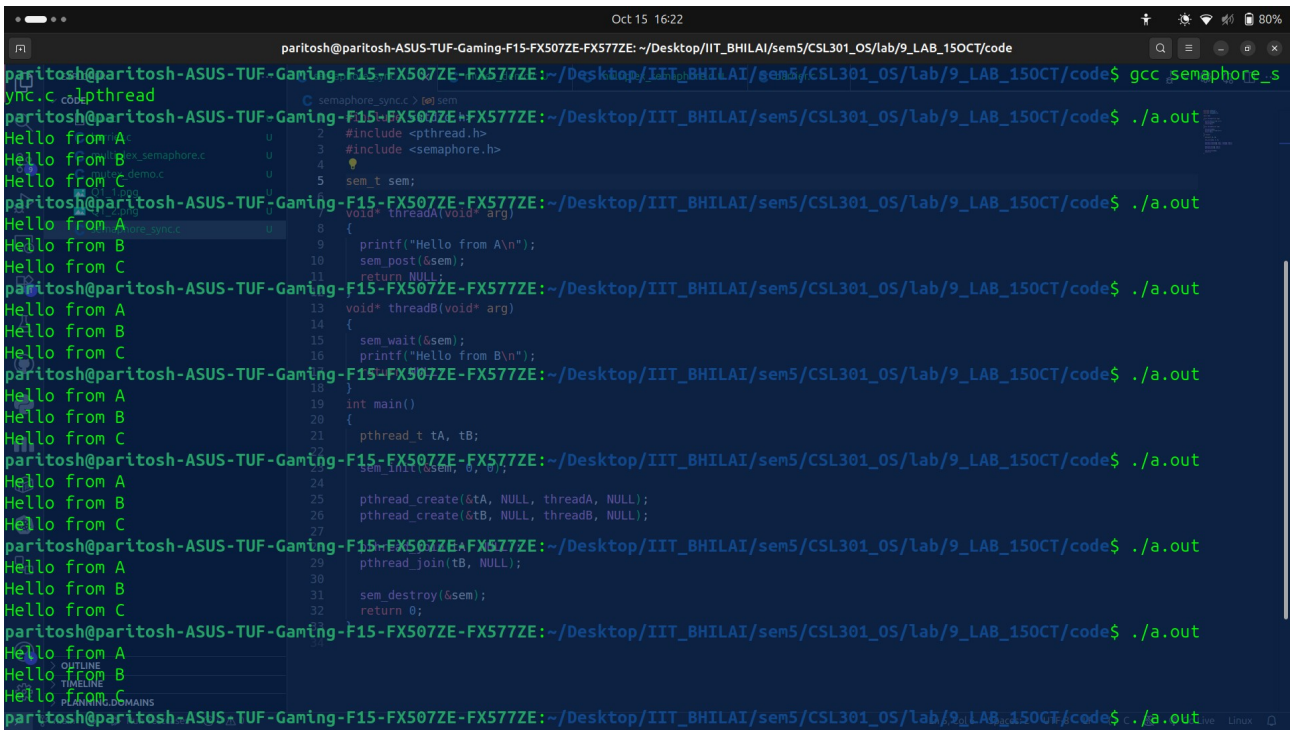
  sem_init(&sem1, 0, 0);
  sem_init(&sem2,0 ,0);

  pthread_create(&tA, NULL, threadA, NULL);
  pthread_create(&tB, NULL, threadB, NULL);

  pthread_join(tA, NULL);
  pthread_join(tB, NULL);

  sem_destroy(&sem1);
  sem_destroy(&sem2);
  return 0;
}
```

ouput:



observation:
here order is maintained A , B, C..due to semaphores.

Q2: Rendezvous Between Two Threads

code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t aArrived, bArrived;


void* threadA(void* arg)
{
printf("A1 (Starting task)\n");
sem_post(&aArrived);
sem_wait(&bArrived);
```

```
printf("A2 (Ending task)\n");
return NULL;
}
void* threadB(void* arg)
{
printf("B1 (Starting task)\n");
sem_post(&bArrived);
sem_wait(&aArrived);
printf("B2 (Ending task)\n");
return NULL;
}

int main()
{
pthread_t tA, tB;

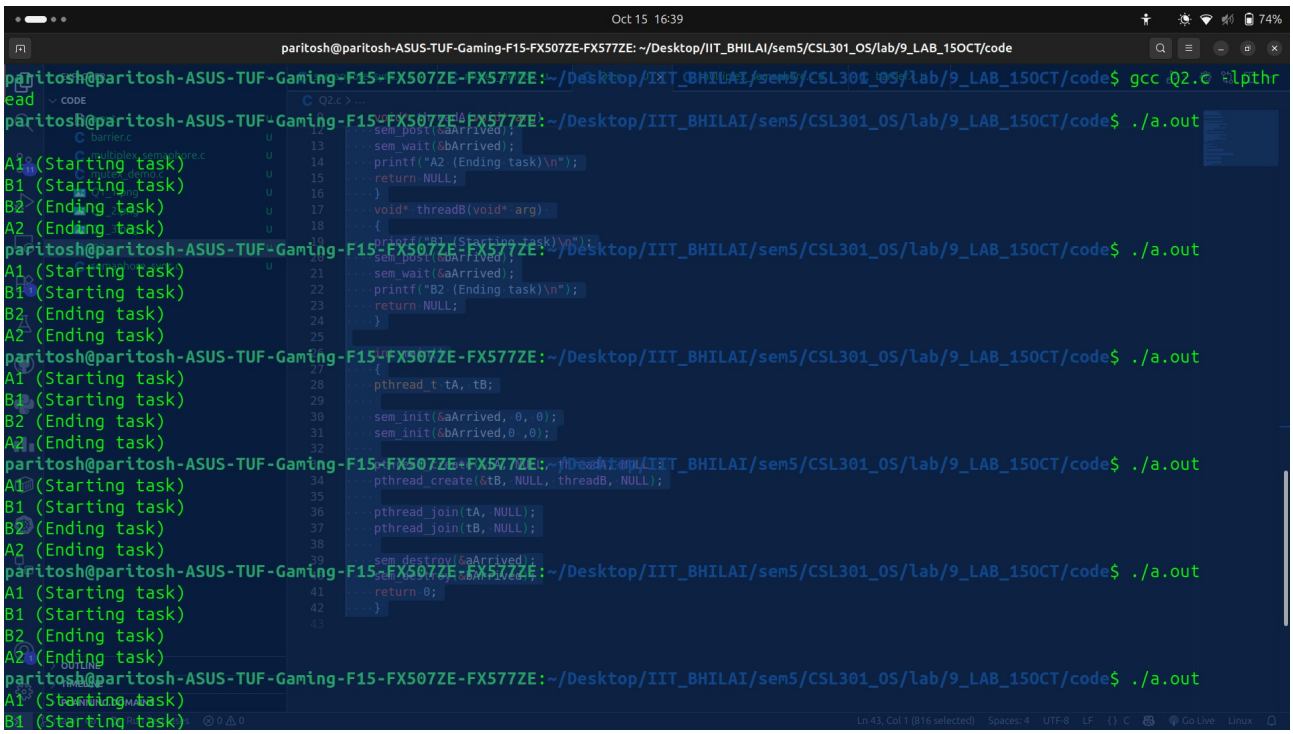
sem_init(&aArrived, 0, 0);
sem_init(&bArrived,0 ,0);

pthread_create(&tA, NULL, threadA, NULL);
pthread_create(&tB, NULL, threadB, NULL);

pthread_join(tA, NULL);
pthread_join(tB, NULL);

sem_destroy(&aArrived);
sem_destroy(&bArrived);
return 0;
}
```

output:



observation:
here we observes rendezvous pattern , its sure that A2 and B2 only execute after A1 and B1..after having synchronixation at a common point.

Q3: Thread Synchronization Using Mutex

code : mutex_demo.c

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* increment(void* arg)
{
  for(int i = 0; i < 5; i++)
  {
```

```
    pthread_mutex_lock(&lock);
    counter = counter + 1;
    printf("Thread %ld: Counter = %d\n", (long)arg, counter);
    pthread_mutex_unlock(&lock);
  }
  return NULL;
}

int main()
{
  pthread_t t1, t2;

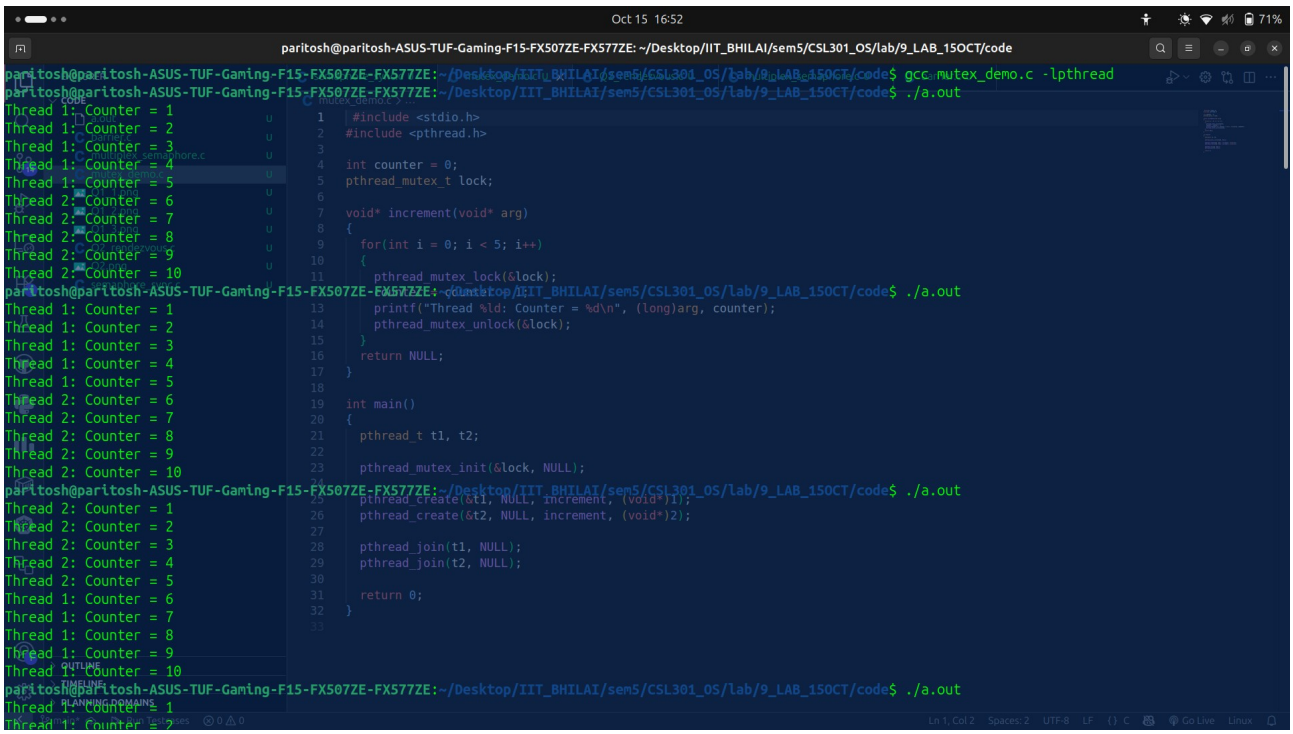
  pthread_mutex_init(&lock, NULL);

  pthread_create(&t1, NULL, increment, (void*)1);
  pthread_create(&t2, NULL, increment, (void*)2);

  pthread_join(t1, NULL);
  pthread_join(t2, NULL);

  return 0;
}
```

output:



Observation:
Here I observe that counter is incremented in order no matter which thread is doing it , mutex ensures no disturbance occur at time of increment.

part2: with mutex
code :

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* increment(void* arg)
{
for(int i = 0; i < 2; i++)
{
    pthread_mutex_lock(&lock);
    counter = counter + 50000;
```

```
        printf("Thread %ld: Counter = %d\n", (long)arg, counter);
        pthread_mutex_unlock(&lock);
    }
    return NULL;
    }

    int main()
    {
    pthread_t arr_t[10];

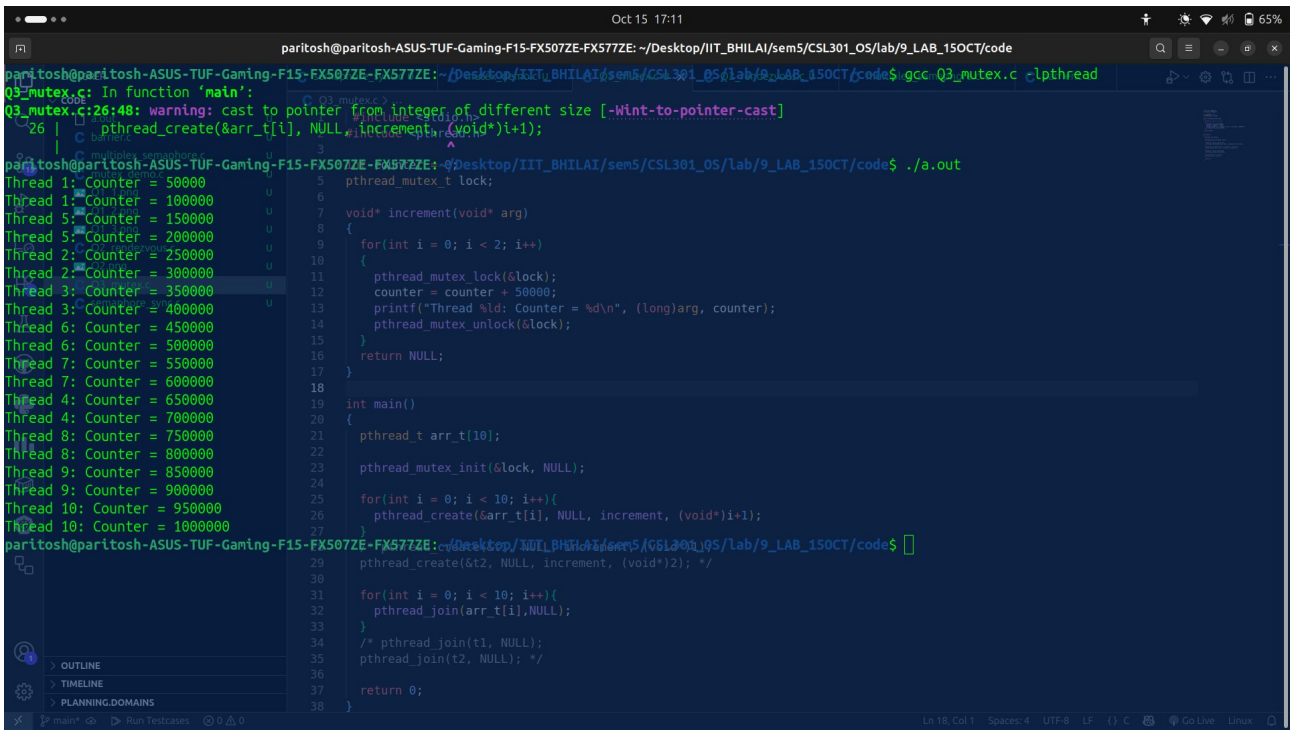
    pthread_mutex_init(&lock, NULL);

    for(int i = 0; i < 10; i++){
        pthread_create(&arr_t[i], NULL, increment, (void*)i+1);
    }
    /* pthread_create(&t1, NULL, increment, (void*)1);
    pthread_create(&t2, NULL, increment, (void*)2); */

    for(int i = 0; i < 10; i++){
        pthread_join(arr_t[i],NULL);
    }
    /* pthread_join(t1, NULL);
    pthread_join(t2, NULL); */

    return 0;
    }
```

output:



observation:
Counter is incremented in order no matter which thread is doing it , mutex ensures no disturbance occur at time of increment.

part3: without mutex

code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void* increment(void* arg)
{
for(int i = 0; i < 2; i++)
{
    counter = counter + 50000;
    printf("Thread %ld: Counter = %d\n", (long)arg, counter);
}
return NULL;
```

```
}

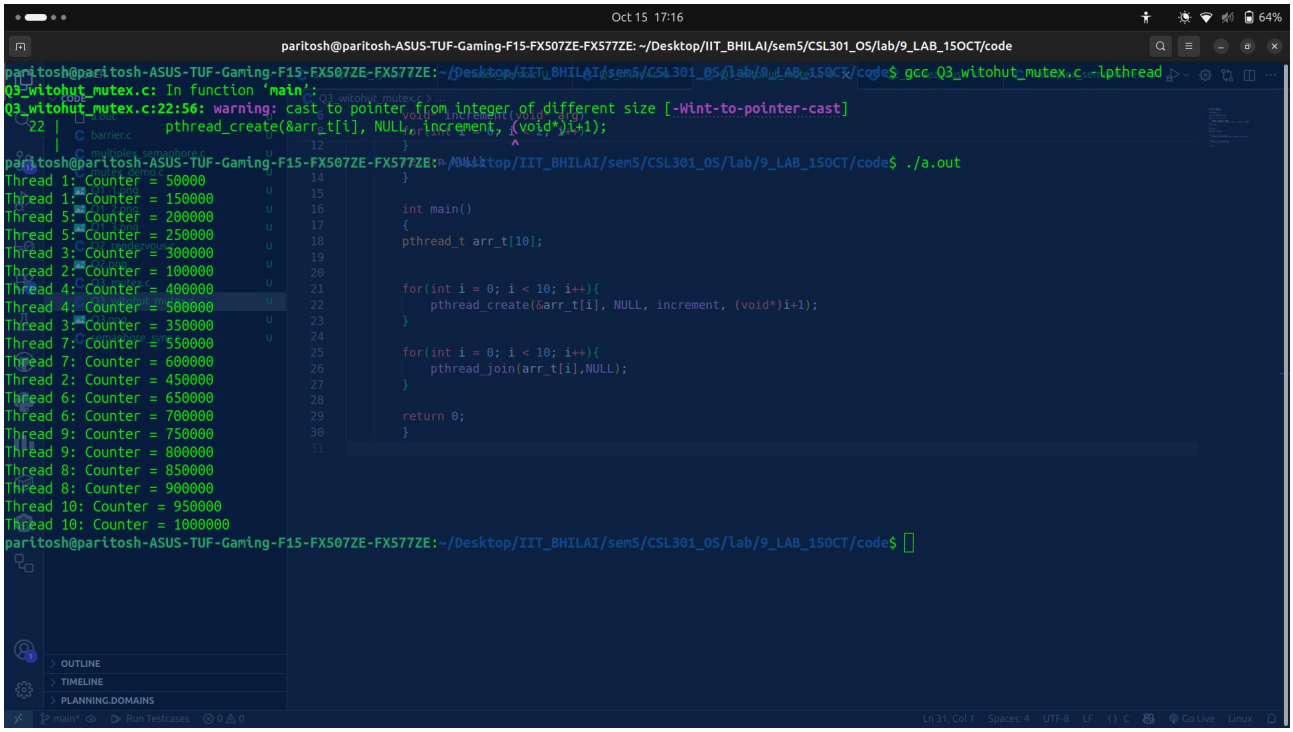
int main()
{
pthread_t arr_t[10];


for(int i = 0; i < 10; i++){
    pthread_create(&arr_t[i], NULL, increment, (void*)i+1);
}

for(int i = 0; i < 10; i++){
    pthread_join(arr_t[i],NULL);
}

return 0;
}
```

output:



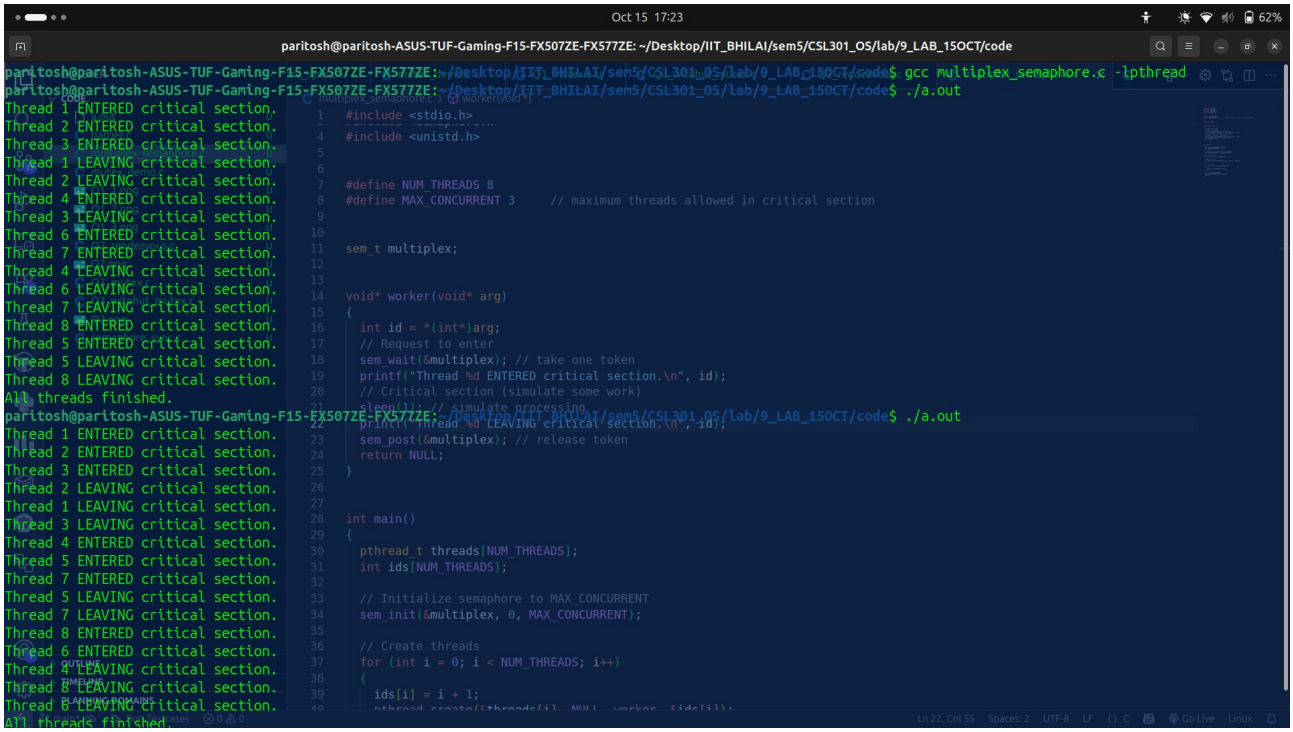
observation:
Here without mutex threads access and modify counter simultaneously but results are inconsistent due to race conditions.

Results are inconsistent due to race conditions.

Q4: Multiplex / Bounded Semaphore

code: multiplex_semaphore.c

output:



Q5: Barrier Synchronization

part A
code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5

int count = 0;
pthread_mutex_t mutex;
sem_t barrier;

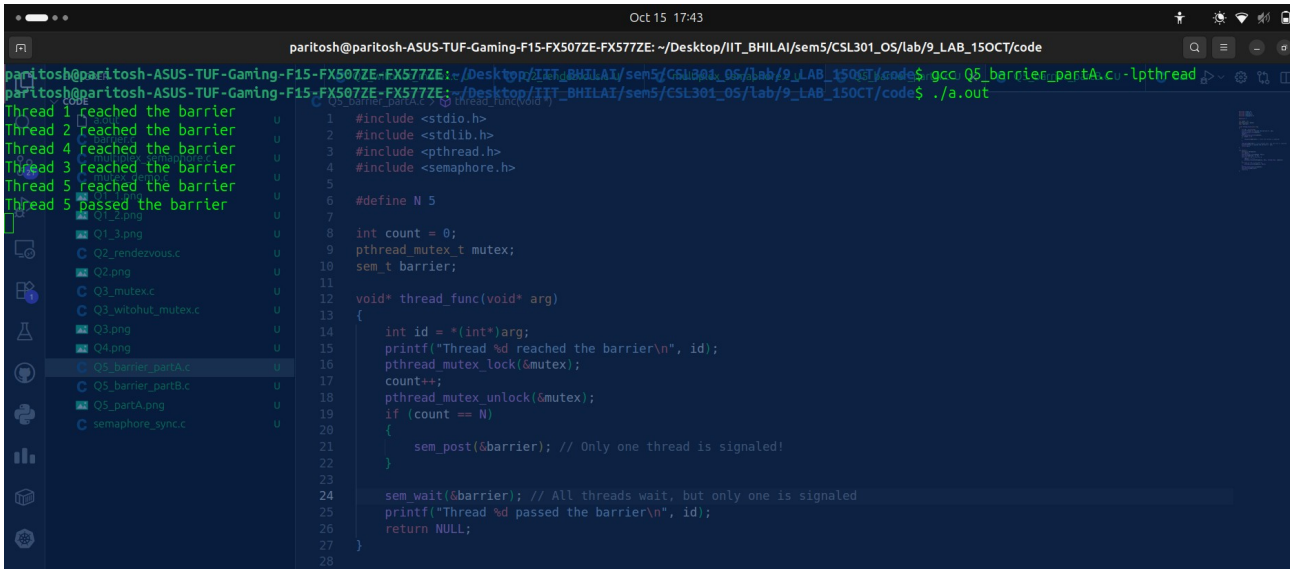
void* thread_func(void* arg)
{
    int id = *(int*)arg;
    printf("Thread %d reached the barrier\n", id);
    pthread_mutex_lock(&mutex);
    count++;
    pthread_mutex_unlock(&mutex);
    if (count == N)
    {
        sem_post(&barrier); // Only one thread is signaled!
    }

    sem_wait(&barrier); // All threads wait, but only one is
signaled
    printf("Thread %d passed the barrier\n", id);
    return NULL;
}

int main() {
    pthread_t threads[N];
    int ids[N];
    pthread_mutex_init(&mutex, NULL);
    sem_init(&barrier, 0, 0);
    for (int i = 0; i < N; i++) {
```

```
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, thread_func, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }
    pthread_mutex_destroy(&mutex);
    sem_destroy(&barrier);
    return 0;
}
```

output:



observation:
Only one thread is released because only one sem_post(&barrier) is done.
Other threads remain blocked at sem_wait(&barrier).
Demonstrates an incorrect barrier implementation.

partB

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5

int count = 0;
pthread_mutex_t mutex;
sem_t turnstile , turnstile2;

void* thread_func(void* arg)
{
    int id = *(int*)arg;
    printf("Thread %d reached the barrier\n", id);
    pthread_mutex_lock(&mutex);
    count++;
    if (count == N)
    {
        sem_wait(&turnstile2);
        sem_post(&turnstile);
    }
    pthread_mutex_unlock(&mutex);
    sem_wait(&turnstile);
    sem_post(&turnstile);
    printf("Thread %d passed the barrier\n", id);
    return NULL;
}

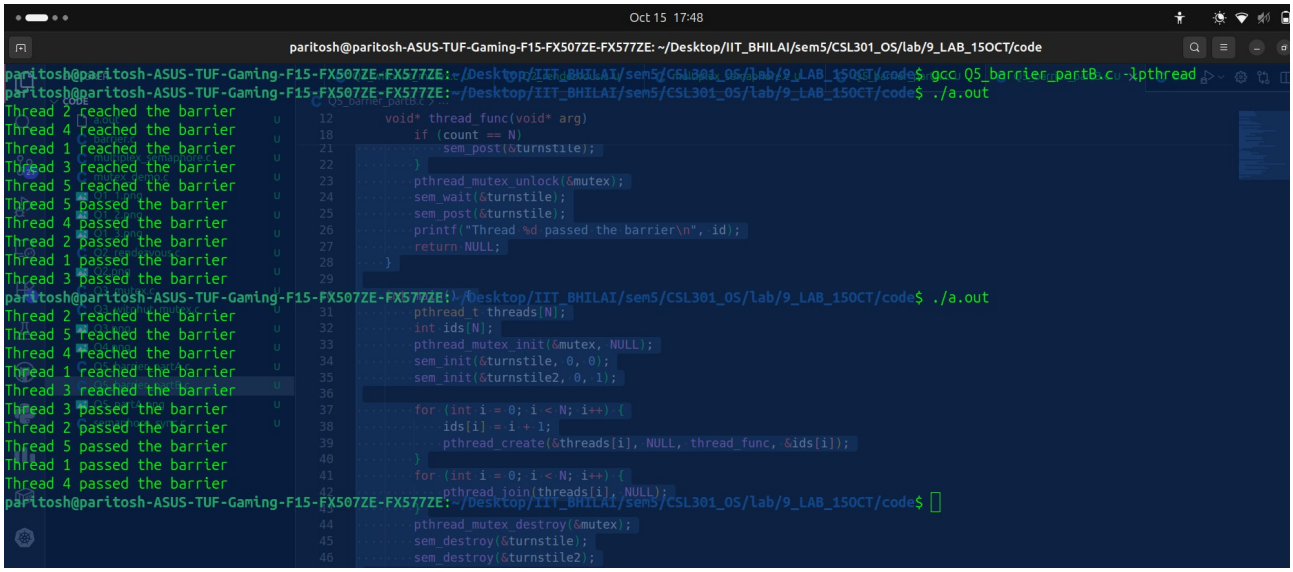
int main() {
    pthread_t threads[N];
```

```
    int ids[N];
    pthread_mutex_init(&mutex, NULL);
    sem_init(&turnstile, 0, 0);
    sem_init(&turnstile2, 0, 1);

    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, thread_func, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }
    pthread_mutex_destroy(&mutex);
    sem_destroy(&turnstile);
    sem_destroy(&turnstile2);

    return 0;
}
```

output:



observation:

All threads reach the barrier → wait until count == N.
Then turnstile opens → all pass the barrier.
Correct barrier synchronization, allowing all threads to proceed only after everyone has reached the barrier point.